Shadow Fleet Detection

Tomas Giedraitis Jonas Adomaitis Gedas Beržinskas

Vilnius University
Faculty of Mathematics and Informatics

2026-04-09

Contents

1. Data
2. Low-Memory Partitioning
3. Shadow Fleet Analytics
4. Speedup Analysis
5. Real-World Geographical Map
6. Architecture

Data

- This dataset contains Danish AIS data from August 13th to 14th, 2025.
- Files processed: `aisdk-2025-08-13.csv` (5.4 GB) and `aisdk-2025-08-14.csv` (5.6 GB).
- Total rows: ∼64 million.

∼26M	15	A · B · C · D
rows/day	**cores**	**anomaly types**
Class A vessels	Shard workers	Detection categories

Low-Memory Partitioning & Dirty Data Handling

Pipeline

1. **csv.reader** line-by-line
   No `pandas.read_csv()` — pure Python generator
2. **parsing.py** filters
   Class A only · invalid MMSI · bad coords
3. **hash(MMSI) % N** shards
   All pings for one ship → same shard file
4. **Flush** buffer when full
   Peak RAM $\leq$ workers $\times$ chunk_size $\times$ $\sim$100 B
5. **Output:** `ais_shard_*.csv`
   15 files on disk, processed in parallel

Dirty Data — Filtered Out

- Non-Class A (Class B, AtoN, Base Station, SAR)
- MMSI 000000000 / 111111111 / 123456789
- All-same-digit MMSIs (unconfigured transponders)
- AIS sentinel coordinates $(91°, 0°)$
- Always-stationary ships (SOG $= 0$ everywhere)

Two-Day Continuity

Both CSVs are processed together.
Shards are sorted chronologically, so gaps spanning midnight

(Aug 13→14) are detected as one continuous

event.

Peak RAM:

$15 \times 50{,}000 \times 100\,\text{B} \approx 75\,\text{MB} \ll 1\,\text{GB/core}$

Shadow Fleet Analytics

A Going Dark

Gap > 4 h + implied speed > 1 kt

Detected in `detect.py` per MMSI shard.

Haversine distance / gap hours = implied speed.

B Loitering / Transfer

Distance < 500 m · SOG < 1 kt · Duration > 2 h

Separate `loiter.py` module.

Bounding-box pre-filter → exact Haversine.

C Draft Changes at Sea

Draught change $> 5\%$ during blackout > 2 h

Compares draught values across AIS gaps.

Indicates illegal cargo transfer at sea.

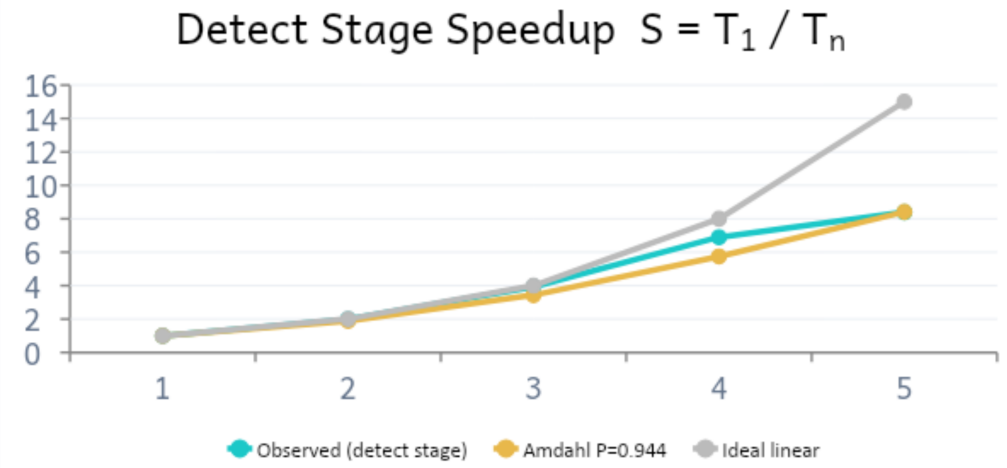
D Identity Cloning

Implied speed > 60 kt · Sustained > 10 min

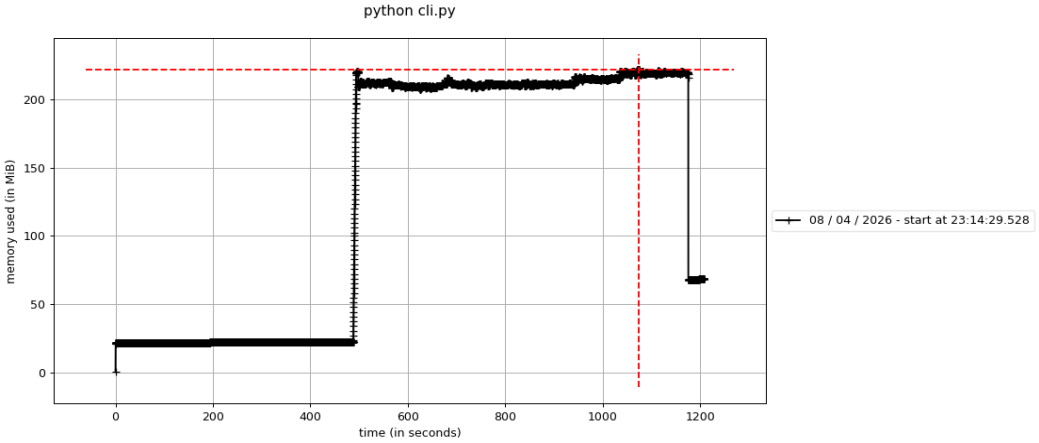
Zigzag pattern across two pings = two ships broadcasting the same stolen MMSI.

DFSI = (Max Gap Hours / 2) + (Largest Impossible Jump NM / 10) + (C $\times$ 15)

Speedup Analysis & Memory Profiling



Overall pipeline: $1.13\times$ | Detect stage: $S = 8.39\times$ | $P \approx 0.944$



245.1 s

Sequential detect
1 worker · 5 153 vessels

29.2 s

Parallel detect
15 workers · `imap_unordered`

$S = 8.39\times$

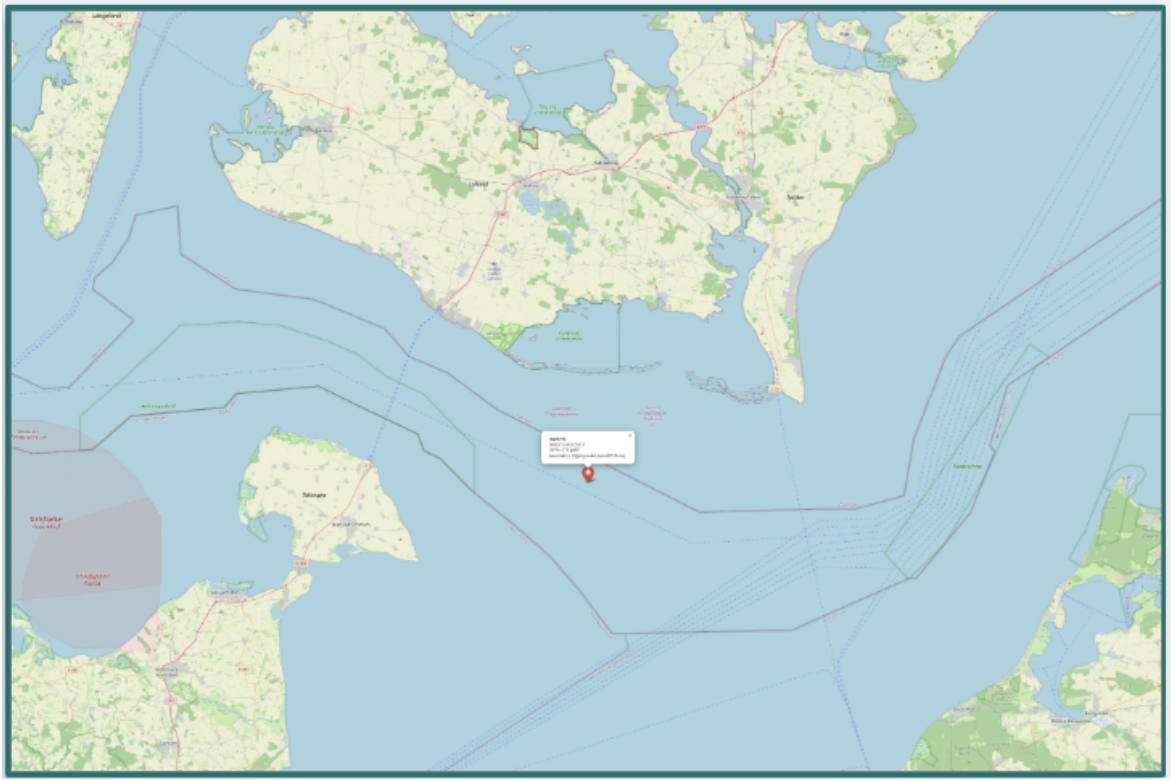
Detect speedup
Overall pipeline $S = 1.13\times$

~220 MiB

Peak RAM
1.4% of 16 GiB limit

Chunk-size: 10k = 1183 s · 50k = 1201 s · 100k = 1213 s
50k chosen as best balance of flush overhead and memory use.

Real-World Geographical Map



#1 Vessel – Top DFSI Score

MMSI	636015357
Ship Name	INVINCIBLE (ZOE T)
Type / Flag	Bulk Carrier · Liberia
DFSI Score	671.50
Anomaly	D – Identity Cloning
Jumps detected	44 impossible pings
Max jump	6 715 nm
Coordinates	54.48°N 11.60°E
Location	Fehmarn Belt, Baltic

Finding: INVINCIBLE broadcasts AIS pings 6 715 nm apart within minutes, which is physically impossible and strongly supports identity cloning.

Architecture

`cli.py` → `pipeline.py` → `partition.py/parsing.py` → `detect.py` (×15) → `loiter.py` → `scoring.py`

`pipeline.py:44`

```
pool.imap_unordered(process_shard,
shard_paths)
```

Distributes shards across Pool workers; results stream back as soon as any worker finishes.

`partition.py:85`

```
_shard_id = hash(mmsi) % num_shards
```

All pings for one MMSI land in the same shard; no cross-shard joins needed.

`detect.py:60`

```
speed = haversine_nm(...) / gap_hours
```

If gap > 4 h but speed > 1 kt, flag Anomaly A.

`loiter.py:72`

```
within_bbox(b.avg_lat, b.avg_lon, bbox)
→ haversine_metres()
```

Bounding-box pre-filter removes distant vessels before exact Haversine check.

Thank you for your time!